

ParamRF: A Modern Framework for Parametric Radio Frequency Modeling

Gary V. C. Allen¹ and Dirk I. L. de Villiers¹

¹ Stellenbosch University, South Africa

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: 

Submitted: 11 June 2026

Published: unpublished

License

Authors of papers retain copyright⁴ and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](https://creativecommons.org/licenses/by/4.0/))⁵.

Summary

ParamRF is an open-source Python framework for the efficient, parametric modeling of radio frequency (RF) circuits and surrogates. The library provides a declarative, object-oriented API, allowing for the composition of deep, nested circuits and the definition of custom models via inheritance. Building on top of the JAX computational ecosystem ([Bradbury et al., 2018](#)), models are represented as pure functions with immutable data alongside. This architecture allows the library to make use of JAX's just-in-time (JIT) compilation to modern hardware (CPU, GPU, TPU), as well as its vectorization and automatic differentiation transformations. Using ParamRF's built-in minimization and Bayesian sampling modules, this enables high-performance optimization, advanced parameter extraction, and new design opportunities.

Statement of Need

The use of radio frequency circuit models is central to fields such as high-frequency electronics, power systems, and semiconductor design. A core engineering requirement in these domains is the creation of parametric circuit models that must be optimized to satisfy specific design goals or fit to measured data for calibration purposes. ParamRF addresses this need by providing researchers and RF engineers with a flexible, programmatic Python framework to easily compose, differentiate, and optimize these circuits.

State of the Field

Currently, researchers and RF engineers often rely on commercial, GUI-driven tools. While powerful, these tools typically lack a flexible programmatic interface, making it difficult to define custom error functions, automate routines, or integrate with modern statistical solvers. Conversely, standard script-based approaches (often built on libraries like NumPy) can introduce significant overhead due to interpreter context switching and dynamic memory allocation. Further, they lack the ability to compute exact analytical gradients, forcing optimizers to rely on slower and approximate numerical differentiation, and also cannot natively encapsulate circuit model uncertainty using Bayesian methods.

Within the open-source community, `scikit-rf` ([Arsenovic et al., 2022](#)) serves as an industry standard for microwave and RF network creation and analysis. However, its primary design is data-driven, storing S-parameter matrices and frequency arrays directly. ParamRF is designed to complement and not replace `scikit-rf` by filling the gap for parameter-focused optimization. It provides an architecture where the core primitives are parametric models as opposed to matrix data containers, allowing the easy creation of complex hierarchical circuits which can be compiled, differentiated and optimized. This approach is not only more intuitive and efficient for optimization purposes, but also provides a foundation for new circuit design opportunities.

39 Models can also optionally be converted to static `scikit-rf` networks after simulation in
40 ParamRF, providing integration with the rest of the Python RF ecosystem.

41 Software Design

42 ParamRF uses a unique approach to circuit modeling via a functional programming paradigm
43 within an object-oriented syntax:

- 44 ▪ **JAX-Native:** The core primitives, `pmrf.Model` and `pmrf.Frequency`, are both JAX
45 PyTrees and Python dataclasses. This means that models are strictly immutable as well
46 as “lazy” i.e., they do not store evaluated matrices, but rather define the pure functions
47 and parameters used to compute their network responses. This provides full compatibility
48 with the JAX ecosystem and therefore the XLA (Accelerated Linear Algebra) compiler.
- 49 ▪ **Declarative Syntax:** Users can define deep, hierarchical structures through a
50 self-documenting syntax. Models can be easily nested and combined using either
51 operator overloading (e.g., using the `**` operator for two-port cascading) or the
52 `pmrf.models.Circuit` class for general circuit topologies. For custom models,
53 `pmrf.Model` can also be overridden where parameters can be given default constraints,
54 catering for more advanced use-cases.
- 55 ▪ **Autodifferentiation:** Because the framework is built on JAX, exact mathematical
56 derivatives of circuit responses (such as S-parameters) can be calculated with respect to
57 both frequency and component parameters. This provides gradient-based solvers with
58 exact Jacobians, greatly improving performance and stability for high-dimensional models.
59 Gradients can also be employed for advanced circuit analysis and design purposes, which
60 is an active area of research.
- 61 ▪ **Parameter Constraints and Manipulation:** Parameters can be scaled, constrained, tied
62 together, or assigned prior probability distributions. This relies on a concept known as
63 “unwrapping”. For example, the built-in factory functions in `pmrf.parameters` return
64 nested wrappers that are unwrapped automatically at evaluation time. This approach
65 provides a bridge between JAX’s pure functional style, and the declarative, object-oriented
66 syntax desired by RF engineers.
- 67 ▪ **Built-in Solvers:** The framework provides high-level sub-modules (`pmrf.optimize`,
68 `pmrf.infer`, `pmrf.fitting`) that abstract different solver backends into a unified
69 interface. Users can easily swap between classical optimization (using SciPy or Optimistix)
70 and Bayesian inference (using BlackJAX) to perform parameter estimation and uncertainty
71 quantification.

72 Research Impact

73 ParamRF was initially developed for the REACH (Radio Experiment for the Analysis of Cosmic
74 Hydrogen, (Lera Acedo et al., 2022)) collaboration. REACH is one of several forefront
75 experiments attempting to detect the cosmological global 21 cm signal (notable others are
76 EDGES (Bowman et al., 2018) and SARAS (Patra et al., 2013)). The project employs physically
77 motivated modeling techniques to perform high-precision calibration of their radiometric
78 instrument, and utilizes ParamRF for the circuit modeling backend in their pipelines. This
79 novel circuit-based calibration approach has been published in the Journal of Astronomical
80 Instrumentation (Allen et al., 2026) and will be presented at the Global 21 cm Workshop
81 2026 (Allen, 2026). Implementing these methods using existing solutions proved inefficient
82 and lacked the capacity for Bayesian uncertainty quantification. By leveraging the automatic
83 differentiation and just-in-time compilation provided by ParamRF, the project achieved between
84 18x and 340x faster optimization of their balun circuit (Allen & De Villiers, 2026). Further,
85 the pipelines rely on the Bayesian solvers natively available in ParamRF, providing a foundation
86 for ongoing statistical research.

87 Although ParamRF originated in radio astronomy, the demand for programmatic and
88 differentiable RF modeling is broad, and these optimization speedups are widely applicable
89 across high-frequency engineering. For instance, recent work on non-uniform amplifier
90 matching networks (Shila, 2025) required implementing design equations directly in Julia
91 to utilize automatic differentiation. ParamRF eliminates the need for such single-use
92 implementations by generalizing this capability to arbitrary circuit topologies. This allows for
93 more efficient optimization, broader design space exploration, and faster calibration workflows
94 directly within Python.

95 Finally, ParamRF significantly lowers the barrier to entry for sensitivity-driven design and
96 microwave circuit uncertainty quantification. While traditional optimizers provide single-point
97 estimates, ParamRF's native integration with Bayesian sampling allows for full design space
98 encapsulation. The ability to compute exact mathematical derivatives, over and above providing
99 exact gradients to solvers, also unlocks exciting new opportunities for advanced circuit analysis
100 and research.

101 Example Usage

102 The following snippet demonstrates ParamRF's syntax and optimization API. A standard RLC
103 (resistor-capacitor-inductor) circuit is first composed using operator overloading. Then, an
104 S_{11} reflection design goal is defined, as well as the target frequency passband over which it
105 should be met. Finally, the goal is minimized, and the initial and optimized models are plotted
106 against each other over a wider frequency range.

```
import pmrf as prf
from pmrf.models import Resistor, Inductor, Capacitor

R = prf.Unconstrained(50.0)
L = prf.Bounded(0.0, 100.0, scale=1e-9)
C = prf.Bounded(0.0, 100.0, scale=1e-12)

model = Resistor(R) ** Inductor(L) ** Capacitor(C)
goal = prf.evaluators.Goal('s11_db', '<', -20)
passband = prf.Frequency(2, 5, 101, 'GHz')

result = prf.optimize.minimize(
    objective=goal,
    model=model,
    frequency=passband,
    solver=prf.optimize.NelderMead()
)

plot_freq = prf.Frequency(1, 6, 101, 'GHz')
model.plot_s_db(plot_freq, m=0, n=0, label='initial')
result.model.plot_s_db(plot_freq, m=0, n=0, label='optimized')
```

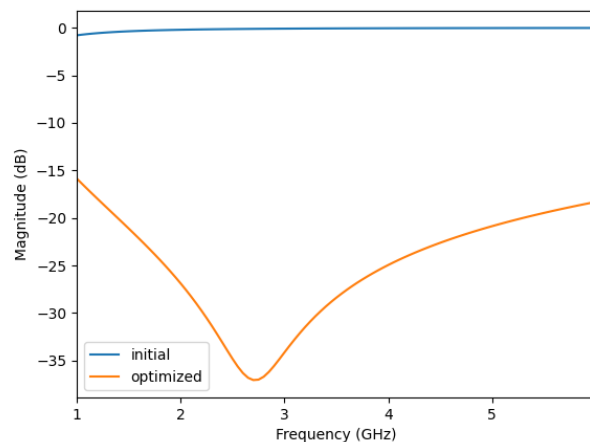


Figure 1: Optimized vs Initial RLC.

AI Usage Disclosure

We disclose the use of generative AI during the development of this project:

- **Tool Identification:** Google Gemini 2.1 and 3.1 Pro was the primary tool used. (Gemini Team, Google, 2023)
- **Scope of Assistance:** The tool was used to automate repetitive tasks, locate and fix bugs, scaffold documentation and tests, and generate the initial implementation for some lower-level models and solver wrappers.
- **Human Review:** All AI-generated content was thoroughly reviewed and validated by the authors. All primary architectural and design decisions were made exclusively by the authors, who bear responsibility for the accuracy and originality of the submitted materials.

Acknowledgements

The research was supported by the South African Radio Astronomy Observatory, which is a facility of the National Research Foundation, an agency of the Department of Science and Technology (Grant Number 75322). The authors would also like to thank the Kavli Foundation, and the Science and Technology Facilities Council, grant number EP/Y02916X1/1, for supporting the REACH project, as well as the developers of JAX, Equinox (Kidger & Garcia, 2021), scikit-rf, and the broader open-source scientific Python community that made this framework possible.

References

- Allen, G. V. C. (2026). *Circuit modelling for precision 21 cm radiometer calibration*. Presentation at the IX Global 21-cm Workshop. <https://global21cmworkshop.org/>
- Allen, G. V. C., & De Villiers, D. (2026). *Efficient balun circuit optimization using accelerated automatic differentiation*.
- Allen, G. V. C., Pegwal, S., De Villiers, D., Anstey, D., Artuc, K., Bevins, H., Bernardi, G., Bucher, M., Carey, S., Cavillot, J., & others. (2026). Circuit Modeling for In Situ 21 cm Radiometer Calibration. *Journal of Astronomical Instrumentation*. <https://doi.org/10.1142/S2251171726500029>
- Arsenovic, A., Hillairet, J., Anderson, J., Forstén, H., Rieß, V., Eller, M., Sauber, N., Weikle, R., Barnhart, W., & Forstmayr, F. (2022). Scikit-rf: An open source python package for microwave network creation, analysis, and calibration [speaker's corner]. *IEEE Microwave Magazine*, 23(1), 98–105. <https://doi.org/10.1109/MMM.2021.3117139>
- Bowman, J. D., Rogers, A. E., Monsalve, R. A., Mozdzen, T. J., & Mahesh, N. (2018). An absorption profile centred at 78 megahertz in the sky-averaged spectrum. *Nature*, 555(7694), 67–70.
- Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S., & Zhang, Q. (2018). *JAX: Composable transformations of Python+NumPy programs* (Version 0.3.13). <http://github.com/google/jax>
- Gemini Team, Google. (2023). Gemini: A Family of Highly Capable Multimodal Models. *arXiv Preprint arXiv:2312.11805*. <https://doi.org/10.26434/chemrxiv-2023-11805>
- Kidger, P., & Garcia, C. (2021). Equinox: Neural networks in JAX via callable PyTrees and filtered transformations. *Differentiable Programming Workshop at Neural Information Processing Systems 2021*. <https://doi.org/10.48550/arXiv.2111.00254>
- Lera Acedo, E. de, Villiers, D. I. L. de, Razavi-Ghods, N., Handley, W., Fialkov, A., Magro, A., Anstey, D., Bevins, H. T. J., Chiello, R., Cumner, J., Josaitis, A. T., Roque, I. L. V., Sims, P. H., Scheutwinkel, K. H., Alexander, P., Bernardi, G., Carey, S., Cavillot, J., Croukamp, W., ... Zarb-Adami, K. (2022). The REACH radiometer for detecting the 21-cm hydrogen signal from redshift $z \sim 7.5$ –28. *Nature Astronomy*, 6(8), 984–998. <https://doi.org/10.1038/s41550-022-01709-9>
- Patra, N., Subrahmanyam, R., Raghunathan, A., & Udaya Shankar, N. (2013). SARAS: A precision system for measurement of the cosmic radio background and signatures from the epoch of reionization. *Experimental Astronomy*, 36(1), 319–370.
- Shila, K. A. (2025). Computationally Efficient Design of an LNA Input Matching Network Using Automatic Differentiation. *IEEE J. Microw.*